MetaOrigin Live Demo Guide and Prompt Framing Experiment

Item	Detail
Prepared for	Iqbal Uddin Khan, Department of IT, FEST, Hamdard University Karachi
Date	10 November 2026
Part 1	MetaOrigin: building, packaging and publishing a small Linux app live (Antigravity and Claude prompts, runbook, backup plan)
Part 2	Prompt framing experiment: one core request, three prompts, with analysis and a controlled version
Companion files	PROMPT_A_Antigravity.txt, PROMPT_B_Claude.txt, BACKUP_install_metaorigin.sh, LIVE_RUNBOOK.txt

Part 1: MetaOrigin live build

1.1 What you will present

Feature	How it works
Metadata and MD5	Reads file metadata and prints MD5 and SHA-256; `--verify` compares with a trusted hash
Edited or original	A scored heuristic: image editor names, XMP history, appended data, PDF incremental updates, revision numbers, tracked changes, extension mismatches
Interfaces	Command line `metaorigin`, and a KDE dialog `metaorigin-gui`
Packaging	A .deb built with `dpkg-deb`; a signed APT repository served from GitHub Pages
Release	Tag v1.0 and a GitHub release created in the browser (no GitHub CLI)
About	Name, designation, company and email from your answers, shown in the app, README, license, web page and package

Important: the verdict is a heuristic. Metadata can be stripped or forged, so the only proof that a file is unchanged is a match with a trusted hash.

1.2 Design decisions and why

Topic	Decision	Reason
Language	Python 3 standard library only	Very limited internet; nothing to install
Python version	Avoid removed modules such as cgi, imghdr, pipes, distutils	Kubuntu 26.04 ships a very new Python
Shell scripts	Strict mode; never pipe into head	A closed pipe can end a strict script with exit code 141
APT layout	Packages in amd64, arm64 and all directories	A package built for all architectures is found on every machine
Repository format	Modern `.sources` file with a key in `/etc/apt/keyrings`	Matches current Ubuntu and Kubuntu practice

Topic	Decision	Reason
Signing key	Created outside the repository, never committed	A private key in Git would be a serious leak
Release	Tag and files prepared by script; publish in the browser	You asked not to use the GitHub CLI
Voice and AI	Not part of this app	Smallest possible scope for a live demo

1.3 What was tested and what was not

Tested in my environment (Ubuntu 24.04, Python 3.12)	Not tested
Reference app passes all 14 self-tests with the sample files	Kubuntu 26.04 itself
The .deb builds, installs and removes cleanly	Antigravity and its settings
The signed APT repository works with real `apt`, over HTTP, as a normal user	The kdialog window (no screen in my environment)
The release files and the v1.0 tag are created	GitHub push, release and Pages (need your account)
The backup installer works from an empty folder	The code the AI writes from the two prompts

1.4 Before the demo checklist

1. Create a blank GitHub repository with no README, license or `.gitignore`.
2. Prepare push access (SSH key or a personal access token) and test it: `git ls-remote https://github.com/USER/REPO.git`.
3. Check the tools: `python3 --version`, `dpkg-deb --version`, `gpg --version`, `git --version`, `command -v kdialog`.
4. Check the clock: `date -u`. APT repository dates use the system clock.
5. In Antigravity set Agent mode to Fast, terminal command auto execution to Turbo or Always Proceed, and the review policy to Always Proceed. Names vary by version, so check right before you paste.
6. Add a deny list for the terminal: sudo, rm -rf, git push, curl, wget, pip, apt.
7. Use the Agent Manager conversation, not inline editor suggestions, so there is nothing to accept.
8. Rehearse once with `bash BACKUP_install_metaorigin.sh` in an empty folder, then delete that folder.

1.5 Live order

1. **Prompt A:** open a new Antigravity conversation in an empty workspace and paste Prompt A. Answer the 6 questions in one message. It should end with ALL TESTS PASSED, a .deb in `dist/` and APT repository check: OK.
2. **Prompt B:** open a new Claude chat, paste Prompt B and answer the 6 questions. Save the reply as `install_metaorigin.sh` and run `bash install_metaorigin.sh` in an empty folder.
3. **Fallback:** if either stalls or the internet drops, run `bash BACKUP_install_metaorigin.sh`. It works offline.
4. **Demo:** run the commands in 1.6, then try a real file such as a phone photo, a PDF or a Word file.

1.6 Demo commands

```
bash scripts/demo.sh
python3 src/metaorigin.py samples/edited.docx
python3 src/metaorigin.py --md5 samples/notes.txt
md5sum samples/notes.txt
python3 src/metaorigin.py --verify HASH samples/notes.txt
```

1.7 Publishing without the GitHub CLI

1. In the project folder run `bash scripts/git_init.sh https://github.com/USER/REPO.git`, then `bash scripts/release_v1.sh`, then `git push -u origin main --tags`.
2. On GitHub open Releases, then Draft a new release. Choose tag v1.0 and set the title to V1.0.
3. Attach the four files from `dist/`: `metaorigin_1.0_all.deb`, `metaorigin-1.0-src.tar.gz`, SHA256SUMS and MD5SUMS. Publish the release.
4. Open Settings, then Pages. Choose Deploy from a branch, branch main, folder /docs, and save.
5. Wait one to two minutes. The page is `https://USER.github.io/REPO/` and the APT repository is `https://USER.github.io/REPO/apt`.

1.8 Install commands for users

```
sudo install -d -m 0755 /etc/apt/keyrings
curl -fsSL https://USER.github.io/REPO/apt/metaorigin.gpg | sudo tee
/etc/apt/keyrings/metaorigin.gpg >/dev/null
printf 'Types: deb\nURIs: https://USER.github.io/REPO/apt\nSuites:
stable\nComponents: main\nSigned-By:
/etc/apt/keyrings/metaorigin.gpg\n' | sudo tee
/etc/apt/sources.list.d/metaorigin.sources
sudo apt update && sudo apt install metaorigin
```

1.9 Troubleshooting

Problem	Fix
Antigravity keeps asking you to accept	Recheck the settings in 1.4 and start a new conversation in Fast mode
Release file is not valid yet	The system clock is wrong; fix it, run `bash scripts/make_apt_repo.sh` again, commit and push
NO_PUBKEY during apt update	The key is not at `/etc/apt/keyrings/metaorigin.gpg` or the Signed-By path is wrong
Pages shows 404	Wait two minutes; confirm Pages uses main and /docs; confirm `docs/apt/dists/stable/InRelease` is in the repository
git push is rejected	The repository was not blank; create a new blank one or run `git pull --rebase`
gpg key creation fails	Check `gpg --version` and that `~/.local/share/metaorigin-gnupg` is writable; without gpg the repository is unsigned
Private key worries	The signing key stays in `~/.local/share/metaorigin-gnupg`; never copy it into the repository

1.10 Prompt A: for Antigravity (paste as is)

```
You are the Antigravity agent in my IDE on Kubuntu 26.04. Build the
complete app described below inside the current workspace folder, in a
new subfolder named metaorigin.

WORKING MODE (important, I am presenting live and will not click
Accept)
- Work in Fast mode. Do NOT write an implementation plan, do NOT create
review artifacts, do NOT wait for any approval.
- Run every terminal command yourself in the integrated terminal. Do
not open a browser, do not search the web.
- Write complete files, not snippets. When a command fails, fix it and
run it again without asking me.
- Keep chat output short: one line per step, then the final report.

STEP 0. Before anything else, ask me these 6 questions in ONE message,
numbered, and wait for my answers. Do not write code or create files
yet.
  1. Creator name (of the pre-built blank repository)
  2. Email
  3. Designation
  4. Company
  5. GitHub username
  6. GitHub repository name (the blank repo I already created)
After I answer, never ask me anything again, never ask for permission
or confirmation, and make every decision yourself.

=== SPECIFICATION ===
PROJECT: MetaOrigin V1.0 (project folder name: metaorigin). One job
only: show a file's metadata, its MD5 (plus SHA-256), and whether it
looks edited or original. Pure Python 3 standard library, CLI plus a
small KDE dialog front end, installable as .deb.

HARD RULES
1. Python 3 standard library only. Target Python 3.10+. Kubuntu 26.04
ships a very new Python, so do NOT use removed modules (cgi, imghdr,
pipes, distutils, telnetlib, pkg_resources). No pip, npm, apt install,
sudo, downloads, web search or network calls: my internet is very
limited.
2. Allowed tools only: python3, bash, git, dpkg-deb, gzip, gpg, md5sum,
sha256sum, tar, stat, kdialog (optional).
3. Every shell script: #!/usr/bin/env bash and set -euo pipefail. Never
pipe a command into head (SIGPIPE breaks pipefail).
4. Read-only: never execute, open for writing or modify an analyzed
file. Hash in 1 MiB chunks. Parse metadata only for files up to 64 MiB
(larger: hashes only).
5. Never run git push, sudo, or rm -rf outside the project folder.
6. Keep it small: src/metaorigin.py at most 450 lines, short comments,
no extra features.
```

```
FOLDER TREE (create exactly this; scripts/bootstrap.sh creates the
folders, makes scripts executable and runs tests/make_samples.py; it
must be safe to run twice)
metaorigin/
  README.md  LICENSE  CHANGELOG.md  .gitignore (build/ dist/
__pycache__/ *.pyc .gnupg/)
  src/metaorigin.py  src/about.json
  packaging/metaorigin  packaging/metaorigin-gui
packaging/metaorigin.desktop  packaging/metaorigin.svg
packaging/make_repo.py
  scripts/bootstrap.sh  build_deb.sh  make_apt_repo.sh
check_apt_local.sh  git_init.sh  release_v1.sh  demo.sh
  tests/make_samples.py  tests/run_tests.sh
  docs/index.html  docs/.nojekyll   (docs/apt is generated)
  samples/  dist/  build/           (generated)


COMMAND LINE (src/metaorigin.py, argparse)
  metaorigin FILE [FILE...]      full report
  --md5          print only "md5  path" lines (same format as md5sum)
  --json         machine-readable report
  --verify HASH  compare with a trusted hash: 32 hex chars = MD5,
otherwise SHA-256; print "Verify   : MATCH" or "MISMATCH"
  --about        print author information from about.json (app,
version, tagline, name, designation, company, email, homepage)
  --version
Exit codes: 0 no edit indicators; 1 edit indicators found or verify
mismatch; 2 error (missing file, directory). Call
signal.signal(SIGPIPE, SIG_DFL) at start of main.


REPORT LABELS (exact): File, MD5, SHA-256, Size, Type, Modified,
Changed, Accessed, Created (only if "stat -c %W" gives a value above
0), Verify (only with --verify), Metadata (key: value lines), Evidence
(lines like "[+3] text"), VERDICT ("VERDICT  : LIKELY EDITED (score
6)"), Note. Times in UTC. MD5 uses hashlib.md5(usedforsecurity=False).
Note text: "Heuristic only: metadata can be stripped or forged. The
only proof a file is unchanged is a match with a trusted hash (use --
verify)."


DETECTION (read magic bytes first; ZIP is OOXML if it has
[Content_Types].xml, ODF if mimetype starts with application/vnd.oasis)
Score = sum of points. Verdict: score >= 3 "LIKELY EDITED"; 1-2
"POSSIBLY EDITED"; 0 and some embedded metadata found "NO EDIT
INDICATORS FOUND"; 0 and no metadata "INCONCLUSIVE". Empty file:
INCONCLUSIVE.
All files: +2 extension does not match the content type (jpg/jpeg, png,
pdf, gif, bmp, zip/docx/xlsx/pptx/odt/ods/odp, exe, mp3, gz); +1
modified time more than 1 day in the future.
PNG (walk chunks): +3 tEXt Software names an editor (photoshop,
lightroom, gimp, paint.net, pixlr, snapseed, canva, affinity, picsart,
facetune, luminar, capture one, acdsee, photoscape, fotor, krita); +3
iTXt XMP (keyword XML:com.adobe.xmp) contains History, DerivedFrom or
```

```
photoshop:; +1 tIME chunk present; +2 bytes after IEND. Show tEXt
fields, image size and tIME as metadata.
JPEG (walk markers; read APP1 Exif little or big endian IFD0 + Exif
IFD, ASCII and SHORT/LONG tags Make, Model, Software, DateTime,
DateTimeOriginal, ExifWidth, ExifHeight): +3 Software names an editor;
+3 XMP contains History, DerivedFrom or photoshop:; +2 DateTime differs
from DateTimeOriginal by more than 2 seconds; +2 Exif dimensions differ
from SOF dimensions; +1 bytes after the final FFD9. No EXIF is shown as
"EXIF: absent" and adds no points.
PDF: count %%EOF; incremental updates = count - 1 (count - 2 if
/Linearized appears in the first 2048 bytes); +3 if updates > 0; read
/Producer /Creator /CreationDate /ModDate /Title /Author with a regex
on the raw bytes; +2 ModDate more than 60 s after CreationDate; +1
Producer or Creator names an editing tool (pdf editor, pdfescape,
sejda, ilovepdf, smallpdf, foxit, pdf-xchange, nitro, pdfelement,
pdffiller, pikepdf, pypdf, pdftk, itext, acrobat).
OOXML (docx, xlsx, pptx): read docProps/core.xml (creator,
lastModifiedBy, created, modified, revision) and docProps/app.xml
(Application, AppVersion, TotalTime): +2 revision > 1; +2 modified more
than 60 s after created; +2 lastModifiedBy differs from creator (both
non-empty); +1 TotalTime > 0; +3 word/document.xml contains "<w:ins "
or "<w:del "; +1 a comments.xml part exists.
ODF (odt, ods, odp): meta.xml: +2 editing-cycles > 1; +2 date more than
60 s after creation-date; +2 creator differs from initial-creator.
Other types: only the generic checks. Parsing errors never crash: show
"Parse note: partial".


SAMPLES (tests/make_samples.py [outdir], stdlib only, deterministic)
and the verdict each must produce
  original.png  8x8 RGB PNG, tEXt Comment only ..... NO EDIT
INDICATORS FOUND
  edited.png    same plus tEXt Software "Adobe Photoshop 25.0", iTXt
XMP with xmpMM:History, tIME chunk, 8 bytes after IEND ... LIKELY
EDITED
  original.pdf  minimal 1-page PDF, Info with Producer (MetaOrigin
Sample), CreationDate=ModDate=D:20260101120000Z, one xref,
one %%EOF ... NO EDIT INDICATORS FOUND
  edited.pdf    original bytes plus a real incremental update: new
object 4 (Info, Producer "Sejda PDF Editor", ModDate
D:20260315093000Z), a new xref section for object 4, trailer with /Prev
(first xref offset), startxref, %%EOF ... LIKELY EDITED
  original.docx zip with [Content_Types].xml, _rels/.rels,
word/document.xml, docProps/core.xml (creator = lastModifiedBy, created
= modified, revision 1) ... NO EDIT INDICATORS FOUND
  edited.docx   revision 7, lastModifiedBy different, modified 6 weeks
later, document.xml contains <w:ins> ... LIKELY EDITED
  camera.jpg    marker-only JPEG (not viewable): SOI, APP1 Exif little-
endian with Make, Model, DateTime, SOF0 8x8, SOS, EOI ... NO EDIT
INDICATORS FOUND
  retouched.jpg same but Software "Adobe Photoshop Lightroom 13" ...
LIKELY EDITED
```

```
  renamed.jpg   the PNG bytes saved with a .jpg name ... POSSIBLY
EDITED
  notes.txt     one line of text ... INCONCLUSIVE
tests/run_tests.sh: build samples in a temp dir; check each verdict
through --json; check --md5 equals md5sum; --verify match exits 0,
mismatch exits 1, missing file exits 2; print PASS/FAIL per check and
"ALL TESTS PASSED"; exit 1 on any failure. scripts/demo.sh runs the
tool on the eight main samples (original and edited pairs, renamed.jpg,
notes.txt) printing MD5, Type, Evidence and VERDICT, then shows --
verify.


FRONT ENDS
packaging/metaorigin: sh script, exec python3
/usr/lib/metaorigin/metaorigin.py "$@".
packaging/metaorigin-gui: bash; loop with kdialog --menu (Check a file
/ About / Quit); kdialog --getopenfilename starting in $HOME; run the
tool, show the report with kdialog --textbox (780x560); About shows the
--about text. If kdialog is missing, print a hint and run the CLI. When
/usr/lib/metaorigin/metaorigin.py does not exist,
use ../src/metaorigin.py relative to the script.
packaging/metaorigin.desktop: Name=MetaOrigin, Exec=metaorigin-gui,
Icon=metaorigin, Terminal=false, Categories=Utility;.
packaging/metaorigin.svg: small SVG icon (document plus magnifier).


DEB (scripts/build_deb.sh, read name, email, version, homepage from
src/about.json with python3, tab separated)
Staging build/metaorigin_1.0_all/: DEBIAN/control,
usr/bin/{metaorigin,metaorigin-gui} (755),
usr/lib/metaorigin/{metaorigin.py,about.json} (644),
usr/share/applications/metaorigin.desktop,
usr/share/icons/hicolor/scalable/apps/metaorigin.svg,
usr/share/doc/metaorigin/{README.md,copyright}. Directories 755.
control: Package: metaorigin; Version: 1.0; Section: utils; Priority:
optional; Architecture: all; Depends: python3 (>= 3.10); Recommends:
kdialog; Maintainer: NAME <EMAIL>; Homepage: URL; Description: one line
plus a short indented paragraph.
Build: dpkg-deb --root-owner-group -Zxz --build STAGING
dist/metaorigin_1.0_all.deb ; finish with dpkg-deb --info to /dev/null
(no head).


APT REPOSITORY for GitHub Pages (scripts/make_apt_repo.sh calls
packaging/make_repo.py DEB docs/apt, stdlib only)
Layout: docs/apt/pool/main/m/metaorigin/metaorigin_1.0_all.deb ;
docs/apt/dists/stable/main/binary-amd64/, binary-arm64/ and binary-all/
each with Packages and Packages.gz (same stanza in all three: output of
dpkg-deb -f plus Filename, Size, MD5sum, SHA256; gzip with mtime=0) ;
docs/apt/dists/stable/Release with Origin, Label, Suite: stable,
Codename: stable, Date (UTC, format "Mon, 09 Nov 2026 22:14:16 UTC"),
Architectures: amd64 arm64 all, Components: main, Description, then
MD5Sum: and SHA256: blocks listing every Packages and Packages.gz as "
hash size path" (path relative to dists/stable, for example
main/binary-amd64/Packages).
```

```
Signing in make_apt_repo.sh: GNUPGHOME defaults to
$HOME/.local/share/metaorigin-gnupg (outside the repo, mode 700). If it
does not exist create a key with: gpg --batch --pinentry-mode loopback
--passphrase '' --quick-generate-key "NAME (MetaOrigin APT) <EMAIL>"
ed25519 sign never. Then: gpg --batch --yes --armor --detach-sign -o
Release.gpg Release; gpg --batch --yes --clearsign -o InRelease
Release; gpg --batch --yes --export -o docs/apt/metaorigin.gpg; same
with --armor to metaorigin.asc. Never commit private keys. touch
docs/.nojekyll.
scripts/check_apt_local.sh (proves the repo with real apt, no root, no
internet): python3 -m http.server on a random port bound to 127.0.0.1 -
-directory docs; temp dir with sources file "deb [signed-
by=TEMP/key.gpg] http://127.0.0.1:PORT/apt stable main" (use
[trusted=yes] only if metaorigin.gpg is missing); run apt-get and apt-
cache with -o Dir::Etc::sourcelist=... -o
Dir::Etc::sourceparts=/dev/null -o Dir::State=TEMP/state -o
Dir::Cache=TEMP/cache -o Debug::NoLocking=1 -o APT::Sandbox::User=$(id
-un) ; create TEMP/state/lists/partial and TEMP/cache/archives/partial
first; run update, policy metaorigin, download metaorigin; clean up
with a trap; print "APT repository check: OK".


GIT AND RELEASE (no GitHub CLI; I push and publish in the browser)
scripts/git_init.sh [REMOTE_URL]: git init -b main if needed; set
repository-local user.name and user.email from about.json; add the
remote when given; git add -A; commit "MetaOrigin V1.0".
scripts/release_v1.sh: require a git repo and a clean tree; git archive
--format=tar.gz --prefix=metaorigin-1.0/ to dist/metaorigin-1.0-
src.tar.gz; create dist/SHA256SUMS and dist/MD5SUMS for the .deb and
the tarball; create annotated tag v1.0 if missing; print the files to
attach and the browser steps.


ABOUT AND DOCS (use the 6 answers everywhere, never placeholders)
src/about.json: app "MetaOrigin", version "1.0", tagline, name,
designation, company, email, homepage https://github.com/GHUSER/GHREPO.
Show the same details in: --about, the GUI About, README.md, LICENSE
(MIT, current year, my name), docs/index.html, the .deb Maintainer, the
git identity.
README.md: what it does, usage, how the verdict works and its limits,
APT install using the modern deb822 format:
    sudo install -d -m 0755 /etc/apt/keyrings
    curl -fsSL https://GHUSER.github.io/GHREPO/apt/metaorigin.gpg |
sudo tee /etc/apt/keyrings/metaorigin.gpg >/dev/null
    printf 'Types: deb\nURIs:
https://GHUSER.github.io/GHREPO/apt\nSuites: stable\nComponents:
main\nSigned-By: /etc/apt/keyrings/metaorigin.gpg\n' | sudo tee
/etc/apt/sources.list.d/metaorigin.sources
    sudo apt update && sudo apt install metaorigin
Use indented code blocks in README.md, never triple backticks.
docs/index.html: one static page, no external resources, dark navy
(#0A0E2A) with cyan (#00F0FF) and violet (#A78BFA), install steps,
links to https://github.com/GHUSER/GHREPO/releases and the repo, an
About card with the author details.
```

```
ORDER OF WORK (after my answers; no more questions, no confirmations)
1 create the tree and files; 2 bash scripts/bootstrap.sh; 3 bash
tests/run_tests.sh and fix until ALL TESTS PASSED (when a test fails
fix the code or the sample, never the expected verdict); 4 bash
scripts/build_deb.sh; 5 bash scripts/make_apt_repo.sh; 6 bash
scripts/check_apt_local.sh; 7 bash scripts/demo.sh; 8 dpkg-deb -c on
the .deb to confirm the file list.
If a command fails: read the error, fix the cause, run it again. After
3 failed attempts on one problem, simplify that part but keep every
file in the tree.

FINAL REPORT (at most 15 lines): test result, .deb size, repo check
result, then my manual steps: (1) bash scripts/git_init.sh
https://github.com/GHUSER/GHREPO.git ; bash scripts/release_v1.sh ; git
push -u origin main --tags ; (2) GitHub web: Releases > Draft a new
release > tag v1.0 > title "V1.0" > attach dist/metaorigin_1.0_all.deb,
dist/metaorigin-1.0-src.tar.gz, dist/SHA256SUMS, dist/MD5SUMS >
Publish ; (3) GitHub web: Settings > Pages > Deploy from a branch >
main > /docs > Save ; (4) after 1-2 minutes run the four install
commands from README.md on a test machine.
```

1.11 Prompt B: for Claude (paste as is)

```
You are Claude. I will run your answer on my Kubuntu 26.04 laptop, and
you cannot create folders there. So you will give me ONE bash script
named install_metaorigin.sh that creates the whole project (every
folder and file in the specification), then runs the tests, builds
the .deb and the APT repository, and runs the demo.

STEP 0. Before anything else, ask me these 6 questions in ONE message,
numbered, and wait for my answers. Do not write code or create files
yet.
  1. Creator name (of the pre-built blank repository)
  2. Email
  3. Designation
  4. Company
  5. GitHub username
  6. GitHub repository name (the blank repo I already created)
After I answer, never ask me anything again, never ask for permission
or confirmation, and make every decision yourself.

AFTER MY ANSWERS, reply with ONLY: one short line, then the script in
ONE code block fenced with FOUR backticks (use ````bash), then a
numbered list of at most 8 lines (how to run it, then my manual GitHub
steps). If you can create files, also attach install_metaorigin.sh. No
web search, no tools needed.

SCRIPT RULES
- Start with #!/usr/bin/env bash and set -euo pipefail. Put my 6
answers at the top as shell variables in single quotes (escape any
```

```
single quote); no prompts and no placeholders anywhere. Create every
folder first with mkdir -p.
- Create each file with cat > path <<'MO_EOF_NN' using a quoted, unique
delimiter per file, then chmod. Use python3 (not sed) for any
substitution of my answers into files, and json.dump for about.json.
- Never put triple backticks inside any file content (the README uses
indented code blocks), so the four-backtick fence stays intact.
- Create all files first, then run steps 2-8 of ORDER OF WORK in
sequence, stop at the first failure with a clear message, and at the
end print the FINAL REPORT manual steps. Support the option --no-run
(create files only).
- My answers replace the placeholders GHUSER, GHREPO and NAME, EMAIL,
DESIGNATION, COMPANY used in the specification.
- If the answer would become too long, shorten comments, README and
index.html, but never drop a file from the tree. Do not write fixes for
problems you cannot see; write the code correctly the first time.


=== SPECIFICATION ===
PROJECT: MetaOrigin V1.0 (project folder name: metaorigin). One job
only: show a file's metadata, its MD5 (plus SHA-256), and whether it
looks edited or original. Pure Python 3 standard library, CLI plus a
small KDE dialog front end, installable as .deb.


HARD RULES
1. Python 3 standard library only. Target Python 3.10+. Kubuntu 26.04
ships a very new Python, so do NOT use removed modules (cgi, imghdr,
pipes, distutils, telnetlib, pkg_resources). No pip, npm, apt install,
sudo, downloads, web search or network calls: my internet is very
limited.
2. Allowed tools only: python3, bash, git, dpkg-deb, gzip, gpg, md5sum,
sha256sum, tar, stat, kdialog (optional).
3. Every shell script: #!/usr/bin/env bash and set -euo pipefail. Never
pipe a command into head (SIGPIPE breaks pipefail).
4. Read-only: never execute, open for writing or modify an analyzed
file. Hash in 1 MiB chunks. Parse metadata only for files up to 64 MiB
(larger: hashes only).
5. Never run git push, sudo, or rm -rf outside the project folder.
6. Keep it small: src/metaorigin.py at most 450 lines, short comments,
no extra features.


FOLDER TREE (create exactly this; scripts/bootstrap.sh creates the
folders, makes scripts executable and runs tests/make_samples.py; it
must be safe to run twice)
metaorigin/
  README.md  LICENSE  CHANGELOG.md  .gitignore (build/ dist/
__pycache__/ *.pyc .gnupg/)
  src/metaorigin.py  src/about.json
  packaging/metaorigin  packaging/metaorigin-gui
packaging/metaorigin.desktop  packaging/metaorigin.svg
packaging/make_repo.py
  scripts/bootstrap.sh  build_deb.sh  make_apt_repo.sh
check_apt_local.sh  git_init.sh  release_v1.sh  demo.sh
```

```
  tests/make_samples.py  tests/run_tests.sh
  docs/index.html  docs/.nojekyll   (docs/apt is generated)
  samples/  dist/  build/           (generated)


COMMAND LINE (src/metaorigin.py, argparse)
  metaorigin FILE [FILE...]      full report
  --md5          print only "md5  path" lines (same format as md5sum)
  --json         machine-readable report
  --verify HASH  compare with a trusted hash: 32 hex chars = MD5,
otherwise SHA-256; print "Verify   : MATCH" or "MISMATCH"
  --about        print author information from about.json (app,
version, tagline, name, designation, company, email, homepage)
  --version
Exit codes: 0 no edit indicators; 1 edit indicators found or verify
mismatch; 2 error (missing file, directory). Call
signal.signal(SIGPIPE, SIG_DFL) at start of main.


REPORT LABELS (exact): File, MD5, SHA-256, Size, Type, Modified,
Changed, Accessed, Created (only if "stat -c %W" gives a value above
0), Verify (only with --verify), Metadata (key: value lines), Evidence
(lines like "[+3] text"), VERDICT ("VERDICT  : LIKELY EDITED (score
6)"), Note. Times in UTC. MD5 uses hashlib.md5(usedforsecurity=False).
Note text: "Heuristic only: metadata can be stripped or forged. The
only proof a file is unchanged is a match with a trusted hash (use --
verify)."


DETECTION (read magic bytes first; ZIP is OOXML if it has
[Content_Types].xml, ODF if mimetype starts with application/vnd.oasis)
Score = sum of points. Verdict: score >= 3 "LIKELY EDITED"; 1-2
"POSSIBLY EDITED"; 0 and some embedded metadata found "NO EDIT
INDICATORS FOUND"; 0 and no metadata "INCONCLUSIVE". Empty file:
INCONCLUSIVE.
All files: +2 extension does not match the content type (jpg/jpeg, png,
pdf, gif, bmp, zip/docx/xlsx/pptx/odt/ods/odp, exe, mp3, gz); +1
modified time more than 1 day in the future.
PNG (walk chunks): +3 tEXt Software names an editor (photoshop,
lightroom, gimp, paint.net, pixlr, snapseed, canva, affinity, picsart,
facetune, luminar, capture one, acdsee, photoscape, fotor, krita); +3
iTXt XMP (keyword XML:com.adobe.xmp) contains History, DerivedFrom or
photoshop:; +1 tIME chunk present; +2 bytes after IEND. Show tEXt
fields, image size and tIME as metadata.
JPEG (walk markers; read APP1 Exif little or big endian IFD0 + Exif
IFD, ASCII and SHORT/LONG tags Make, Model, Software, DateTime,
DateTimeOriginal, ExifWidth, ExifHeight): +3 Software names an editor;
+3 XMP contains History, DerivedFrom or photoshop:; +2 DateTime differs
from DateTimeOriginal by more than 2 seconds; +2 Exif dimensions differ
from SOF dimensions; +1 bytes after the final FFD9. No EXIF is shown as
"EXIF: absent" and adds no points.
PDF: count %%EOF; incremental updates = count - 1 (count - 2 if
/Linearized appears in the first 2048 bytes); +3 if updates > 0; read
/Producer /Creator /CreationDate /ModDate /Title /Author with a regex
on the raw bytes; +2 ModDate more than 60 s after CreationDate; +1
```

```
Producer or Creator names an editing tool (pdf editor, pdfescape,
sejda, ilovepdf, smallpdf, foxit, pdf-xchange, nitro, pdfelement,
pdffiller, pikepdf, pypdf, pdftk, itext, acrobat).
OOXML (docx, xlsx, pptx): read docProps/core.xml (creator,
lastModifiedBy, created, modified, revision) and docProps/app.xml
(Application, AppVersion, TotalTime): +2 revision > 1; +2 modified more
than 60 s after created; +2 lastModifiedBy differs from creator (both
non-empty); +1 TotalTime > 0; +3 word/document.xml contains "<w:ins "
or "<w:del "; +1 a comments.xml part exists.
ODF (odt, ods, odp): meta.xml: +2 editing-cycles > 1; +2 date more than
60 s after creation-date; +2 creator differs from initial-creator.
Other types: only the generic checks. Parsing errors never crash: show
"Parse note: partial".


SAMPLES (tests/make_samples.py [outdir], stdlib only, deterministic)
and the verdict each must produce
  original.png  8x8 RGB PNG, tEXt Comment only ..... NO EDIT
INDICATORS FOUND
  edited.png    same plus tEXt Software "Adobe Photoshop 25.0", iTXt
XMP with xmpMM:History, tIME chunk, 8 bytes after IEND ... LIKELY
EDITED
  original.pdf  minimal 1-page PDF, Info with Producer (MetaOrigin
Sample), CreationDate=ModDate=D:20260101120000Z, one xref,
one %%EOF ... NO EDIT INDICATORS FOUND
  edited.pdf    original bytes plus a real incremental update: new
object 4 (Info, Producer "Sejda PDF Editor", ModDate
D:20260315093000Z), a new xref section for object 4, trailer with /Prev
(first xref offset), startxref, %%EOF ... LIKELY EDITED
  original.docx zip with [Content_Types].xml, _rels/.rels,
word/document.xml, docProps/core.xml (creator = lastModifiedBy, created
= modified, revision 1) ... NO EDIT INDICATORS FOUND
  edited.docx   revision 7, lastModifiedBy different, modified 6 weeks
later, document.xml contains <w:ins> ... LIKELY EDITED
  camera.jpg    marker-only JPEG (not viewable): SOI, APP1 Exif little-
endian with Make, Model, DateTime, SOF0 8x8, SOS, EOI ... NO EDIT
INDICATORS FOUND
  retouched.jpg same but Software "Adobe Photoshop Lightroom 13" ...
LIKELY EDITED
  renamed.jpg   the PNG bytes saved with a .jpg name ... POSSIBLY
EDITED
  notes.txt     one line of text ... INCONCLUSIVE
tests/run_tests.sh: build samples in a temp dir; check each verdict
through --json; check --md5 equals md5sum; --verify match exits 0,
mismatch exits 1, missing file exits 2; print PASS/FAIL per check and
"ALL TESTS PASSED"; exit 1 on any failure. scripts/demo.sh runs the
tool on the eight main samples (original and edited pairs, renamed.jpg,
notes.txt) printing MD5, Type, Evidence and VERDICT, then shows --
verify.


FRONT ENDS
packaging/metaorigin: sh script, exec python3
/usr/lib/metaorigin/metaorigin.py "$@".
```

```
packaging/metaorigin-gui: bash; loop with kdialog --menu (Check a file
/ About / Quit); kdialog --getopenfilename starting in $HOME; run the
tool, show the report with kdialog --textbox (780x560); About shows the
--about text. If kdialog is missing, print a hint and run the CLI. When
/usr/lib/metaorigin/metaorigin.py does not exist,
use ../src/metaorigin.py relative to the script.
packaging/metaorigin.desktop: Name=MetaOrigin, Exec=metaorigin-gui,
Icon=metaorigin, Terminal=false, Categories=Utility;.
packaging/metaorigin.svg: small SVG icon (document plus magnifier).

DEB (scripts/build_deb.sh, read name, email, version, homepage from
src/about.json with python3, tab separated)
Staging build/metaorigin_1.0_all/: DEBIAN/control,
usr/bin/{metaorigin,metaorigin-gui} (755),
usr/lib/metaorigin/{metaorigin.py,about.json} (644),
usr/share/applications/metaorigin.desktop,
usr/share/icons/hicolor/scalable/apps/metaorigin.svg,
usr/share/doc/metaorigin/{README.md,copyright}. Directories 755.
control: Package: metaorigin; Version: 1.0; Section: utils; Priority:
optional; Architecture: all; Depends: python3 (>= 3.10); Recommends:
kdialog; Maintainer: NAME <EMAIL>; Homepage: URL; Description: one line
plus a short indented paragraph.
Build: dpkg-deb --root-owner-group -Zxz --build STAGING
dist/metaorigin_1.0_all.deb ; finish with dpkg-deb --info to /dev/null
(no head).

APT REPOSITORY for GitHub Pages (scripts/make_apt_repo.sh calls
packaging/make_repo.py DEB docs/apt, stdlib only)
Layout: docs/apt/pool/main/m/metaorigin/metaorigin_1.0_all.deb ;
docs/apt/dists/stable/main/binary-amd64/, binary-arm64/ and binary-all/
each with Packages and Packages.gz (same stanza in all three: output of
dpkg-deb -f plus Filename, Size, MD5sum, SHA256; gzip with mtime=0) ;
docs/apt/dists/stable/Release with Origin, Label, Suite: stable,
Codename: stable, Date (UTC, format "Mon, 09 Nov 2026 22:14:16 UTC"),
Architectures: amd64 arm64 all, Components: main, Description, then
MD5Sum: and SHA256: blocks listing every Packages and Packages.gz as "
hash size path" (path relative to dists/stable, for example
main/binary-amd64/Packages).
Signing in make_apt_repo.sh: GNUPGHOME defaults to
$HOME/.local/share/metaorigin-gnupg (outside the repo, mode 700). If it
does not exist create a key with: gpg --batch --pinentry-mode loopback
--passphrase '' --quick-generate-key "NAME (MetaOrigin APT) <EMAIL>"
ed25519 sign never. Then: gpg --batch --yes --armor --detach-sign -o
Release.gpg Release; gpg --batch --yes --clearsign -o InRelease
Release; gpg --batch --yes --export -o docs/apt/metaorigin.gpg; same
with --armor to metaorigin.asc. Never commit private keys. touch
docs/.nojekyll.
scripts/check_apt_local.sh (proves the repo with real apt, no root, no
internet): python3 -m http.server on a random port bound to 127.0.0.1 -
-directory docs; temp dir with sources file "deb [signed-
by=TEMP/key.gpg] http://127.0.0.1:PORT/apt stable main" (use
[trusted=yes] only if metaorigin.gpg is missing); run apt-get and apt-
```

```
cache with -o Dir::Etc::sourcelist=... -o
Dir::Etc::sourceparts=/dev/null -o Dir::State=TEMP/state -o
Dir::Cache=TEMP/cache -o Debug::NoLocking=1 -o APT::Sandbox::User=$(id
-un) ; create TEMP/state/lists/partial and TEMP/cache/archives/partial
first; run update, policy metaorigin, download metaorigin; clean up
with a trap; print "APT repository check: OK".

GIT AND RELEASE (no GitHub CLI; I push and publish in the browser)
scripts/git_init.sh [REMOTE_URL]: git init -b main if needed; set
repository-local user.name and user.email from about.json; add the
remote when given; git add -A; commit "MetaOrigin V1.0".
scripts/release_v1.sh: require a git repo and a clean tree; git archive
--format=tar.gz --prefix=metaorigin-1.0/ to dist/metaorigin-1.0-
src.tar.gz; create dist/SHA256SUMS and dist/MD5SUMS for the .deb and
the tarball; create annotated tag v1.0 if missing; print the files to
attach and the browser steps.

ABOUT AND DOCS (use the 6 answers everywhere, never placeholders)
src/about.json: app "MetaOrigin", version "1.0", tagline, name,
designation, company, email, homepage https://github.com/GHUSER/GHREPO.
Show the same details in: --about, the GUI About, README.md, LICENSE
(MIT, current year, my name), docs/index.html, the .deb Maintainer, the
git identity.
README.md: what it does, usage, how the verdict works and its limits,
APT install using the modern deb822 format:
    sudo install -d -m 0755 /etc/apt/keyrings
    curl -fsSL https://GHUSER.github.io/GHREPO/apt/metaorigin.gpg |
sudo tee /etc/apt/keyrings/metaorigin.gpg >/dev/null
    printf 'Types: deb\nURIs:
https://GHUSER.github.io/GHREPO/apt\nSuites: stable\nComponents:
main\nSigned-By: /etc/apt/keyrings/metaorigin.gpg\n' | sudo tee
/etc/apt/sources.list.d/metaorigin.sources
    sudo apt update && sudo apt install metaorigin
Use indented code blocks in README.md, never triple backticks.
docs/index.html: one static page, no external resources, dark navy
(#0A0E2A) with cyan (#00F0FF) and violet (#A78BFA), install steps,
links to https://github.com/GHUSER/GHREPO/releases and the repo, an
About card with the author details.

ORDER OF WORK (after my answers; no more questions, no confirmations)
1 create the tree and files; 2 bash scripts/bootstrap.sh; 3 bash
tests/run_tests.sh and fix until ALL TESTS PASSED (when a test fails
fix the code or the sample, never the expected verdict); 4 bash
scripts/build_deb.sh; 5 bash scripts/make_apt_repo.sh; 6 bash
scripts/check_apt_local.sh; 7 bash scripts/demo.sh; 8 dpkg-deb -c on
the .deb to confirm the file list.
If a command fails: read the error, fix the cause, run it again. After
3 failed attempts on one problem, simplify that part but keep every
file in the tree.
```

```
FINAL REPORT (at most 15 lines): test result, .deb size, repo check
result, then my manual steps: (1) bash scripts/git_init.sh
https://github.com/GHUSER/GHREPO.git ; bash scripts/release_v1.sh ; git
push -u origin main --tags ; (2) GitHub web: Releases > Draft a new
release > tag v1.0 > title "V1.0" > attach dist/metaorigin_1.0_all.deb,
dist/metaorigin-1.0-src.tar.gz, dist/SHA256SUMS, dist/MD5SUMS >
Publish ; (3) GitHub web: Settings > Pages > Deploy from a branch >
main > /docs > Save ; (4) after 1-2 minutes run the four install
commands from README.md on a test machine.
```

1.12 Backup installer

BACKUP_install_metaorigin.sh is a single script that asks the same 6 questions, creates the whole project, runs the tests, builds the .deb and the APT repository, checks the repository with real apt, and runs the demo. Run it with --no-run to create the files only.

```
bash BACKUP_install_metaorigin.sh
bash BACKUP_install_metaorigin.sh --no-run
```

Part 2: Prompt framing experiment

2.1 Purpose and setup

The experiment shows that the way a request is framed changes the answer, even when the underlying goal is the same. The shared goal is: **how to overcome procrastination and manage time better.** Open three separate, fresh Gemini chat windows and paste one prompt into each. The same method works with any large language model.

1. Use the same model and mode in all three chats and note the model name and date.
2. Start each chat fresh so earlier messages cannot influence the answer.
3. If personalization or memory features are on, switch them off or use a temporary chat if available.
4. Paste exactly one prompt per chat and do not add follow-up messages.
5. Record the result with the sheet in 2.7.

2.2 The three prompts

Chat 1: The tactical execution plan

```
I keep procrastinating on my work tasks. Give me a strict, step-by-step
action plan for tomorrow morning to get straight to work. Keep it
concise, practical, and focused entirely on immediate execution without
long explanations.
```

Chat 2: The conversational pep talk

```
I'm having a hard time focusing and keep putting things off today. Talk
to me like a supportive friend over coffee–give me a lighthearted pep
talk, use a fun analogy to explain why my brain is doing this, and
share 2 simple mindset shifts to help me start.
```

Chat 3: The analytical and structural comparison

```
Analyze procrastination from a productivity framework perspective.
Provide a comparison table of 3 popular time-management systems (e.g.,
Time Blocking, Pomodoro, Eat the Frog), detailing the core mechanics,
pros, cons, and ideal user profile for each.
```

2.3 What each chat will probably produce

These are predictions from how prompts usually steer models, not guaranteed results. Models vary between runs, so treat a single run as an illustration, not proof.

Chat	Tone and style	Format	Lesson about the model
1	Direct, urgent, actionable	Numbered steps and a checklist	Constraints such as 'strict' and 'without long explanations' push the model to skip theory
2	Warm, empathetic, humorous	Conversational prose	A persona and context ('friend over coffee') change vocabulary, tone and pacing
3	Objective, analytical	A table with category breakdowns	A requested format ('comparison table') organizes the answer and adds depth

2.4 Analysis: what each prompt actually does

Mapping each prompt to the six parts of a strong prompt (role, context, task, constraints, format, examples) shows which levers are doing the work.

Part	Chat 1	Chat 2	Chat 3
Role	None stated	Supportive friend	Productivity analyst (implied)
Context	Work tasks, tomorrow morning	Struggling to focus today	Productivity frameworks
Task	Give an action plan	Pep talk, analogy, 2 mindset shifts	Compare 3 systems
Constraints	Strict, concise, no long explanations	Lighthearted; exactly 2 shifts	Exactly 3 systems; four named attributes
Format	Step-by-step plan (implied list)	Prose (implied)	Comparison table (explicit)
Examples	None	None	Names Time Blocking, Pomodoro, Eat the Frog

Observation 1: the three prompts are not the same request. Chat 1 asks for a plan for tomorrow morning, Chat 2 responds to a feeling today, and Chat 3 asks for a comparison of systems. Because the task and the time frame also differ, the outputs differ for more than framing reasons. This is a good demonstration of how prompts steer answers, but it is not a controlled test of one lever.

Observation 2: Chat 3 adds knowledge, not only tone. It names three systems and four attributes, so the model is told what content to produce. Chat 1 and Chat 2 leave the content mostly open.

Observation 3: counts are strong constraints. 'Exactly 2 mindset shifts' and 'exactly 3 systems' are easy to check, which makes them useful for grading.

Observation 4: Chat 3 should be fact-checked. A table looks authoritative. Check the claims, for example that Pomodoro uses focused work intervals (traditionally 25 minutes) separated by short breaks, that Eat the Frog means doing the hardest task first, and that Time Blocking assigns tasks to scheduled calendar slots.

2.5 The levers of prompting

The experiment teaches three levers. A fourth, context, is worth adding because it explains why Chat 1 and Chat 2 differ in length and focus.

Lever	Question it answers	Example in these chats	Typical effect
Role or tone	Who is speaking?	Friend over coffee	Changes vocabulary, warmth and pacing
Format	What should the answer look like?	Comparison table	Changes structure and depth
Constraints	What must it include or avoid?	No long explanations; exactly 2 shifts	Changes length and scope
Context and audience	Who is it for and in what situation?	Tomorrow morning; work tasks	Changes relevance and detail

2.6 A controlled version of the experiment

To isolate one lever at a time, keep the same task and change only one thing. Run all five in fresh chats and compare.

Chat	Variation	Prompt
A	Baseline	How can I overcome procrastination and manage my time better?
B	Role added	You are a supportive friend. How can I overcome procrastination and manage my time better?
C	Format added	How can I overcome procrastination and manage my time better? Answer as a comparison table of 3 methods with pros and cons.
D	Constraints added	How can I overcome procrastination and manage my time better? Give exactly 5 steps, each under 15 words, and no introduction.
E	All levers	You are a supportive friend. I procrastinate on work tasks. Give exactly 5 steps for tomorrow morning, each under 15 words, as a numbered list, with no introduction.

Chat A shows the model's default. Chats B, C and D each show one lever. Chat E shows the combined effect and usually gives the most usable result.

2.7 Recording sheet and scoring rubric

Score each answer from 1 (poor) to 5 (excellent) and record the answer length in words.

Chat	Words	Tone fits the prompt	Format fits the prompt	Actionable	Facts correct	Notes
1						
2						
3						
A to E						

Criterion	What a score of 5 means
Tone fits	The voice matches the role and audience requested
Format fits	The structure matches what was asked (list, prose, table) with no extra sections
Actionable	You could start the first step immediately
Facts correct	Every factual claim checks out against a reliable source
Constraint followed	Counts, length limits and exclusions are respected exactly

2.8 Pitfalls to control

- Different model versions or modes give different answers; keep them the same and record them.
- Run each prompt twice. If the two runs differ a lot, say so; one run is not proof.
- Do not rewrite a prompt after seeing the answer unless you note it as a new version.
- Avoid personal or sensitive details in the prompts.
- Do not treat a confident, well-formatted answer as a correct answer; verify the facts.

2.9 How to explain this experiment to users

1. **Same goal, different angle.** All three chats address procrastination, but each prompt turns the model into a different tool: a checklist maker, a coach or a researcher.
2. **Show the three levers.** Role or tone (who is speaking), format (what the answer looks like) and constraints (what to include or avoid). Add context as the fourth.
3. **Compare side by side.** Put the three answers on one screen and ask the audience which one they would act on, and why.
4. **Be honest about control.** Explain that the three prompts also differ in task, so the controlled version in 2.6 is the fair test of each lever.
5. **Connect to the project.** The MedAgent prompt in the PROMPTOBIA guide uses all levers: role, context, task, constraints and format.

2.10 Discussion questions

- Which lever changed the answer most, and was that a surprise?
- When is a table the wrong format for the audience?
- How would you check Chat 3 for errors?
- What constraint would you add to make Chat 2 shorter without losing the analogy?